

Laboratorium 10: XAI

Zadanie 1: Permutation feature importance dla „toy set”

Cel: Implementacja kodu do oceny istotności cech klasyfikatora wieloklasowego z wykorzystaniem „permutation feature importance”

Studenci implementują:

1. Przygotowanie zbioru „toy set” oraz podział na części train_val i testową

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

# Data
X, y = make_classification(n_samples=5000, n_features=20, n_classes=4, n_informative = 4,
                          random_state=42)
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.long)

X_train_val, X_test, y_train_val, y_test = train_test_split(X, y, test_size=0.2)
```

2. Trening sieci do klasyfikacji w schemacie walidacji krzyżowej, zapisanie najlepszych modeli na zbiorach walidacyjnych

3. Implementacja wrappera do modelu klasyfikatora pytorch (identycznie, jak Zadanie 3 w Zestawie 9)

4. Implementacja kodu do oceny istotności cech modeli z ensemble z pkt. 2:

```
from sklearn.inspection import permutation_importance
r = permutation_importance(clf, X_val, y_val, n_repeats=100, random_state=0)
```

4. Selekcja cech istotnych na poziomie istotności 5% (95%-owy przedział ufności, wyznaczony dla `n_repeats` nie zawiera zera) – odrębnie dla zbioru treningowego i testowego. Czy wśród cech istotnych są takie, dla których wartość „importance” jest mniejsza od zera? Jakie konsekwencje dla modelu mają cechy o wartości „importance” mniejszej od zera?

5. Wizualizacja rozkładu wag cech.

Zadanie: Wykonaj ocenę istotności cech metodą „permutation feature importance” w oparciu o zbiór „toy set”.

Zadanie 2: Partial dependency plots (PDP) dla „toy set”

Cel: Wyznaczenie modelu wpływu cechy na odpowiedź regresora

Niech X_s będzie cechą, której wpływ na odpowiedź modelu badamy, a X_c niech będzie zbiorem wszystkich cech z wyjątkiem X_s . „Partial response” modelu f dla $X_s = x_s$ jest dane wzorem:

$$pd_{X_S}(x_S) \stackrel{def}{=} \mathbb{E}_{X_C} [f(x_S, X_C)]$$

$$= \int f(x_S, x_C) p(x_C) dx_C$$

„Partial dependency plot” to wykres $pd_{X_S}(x_S)$ od x_S , „Individual conditional expectation” to wykres $f(x_S, x_C^{(i)})$ od x_S .

Studenci implementują:

1. Przygotowanie zbioru „toy set” dla zadania regresji oraz podział na części train_val i testową:

```
import torch
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split

# Data
X, y = make_regression(n_samples=5000, n_features=20, random_state=42)
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32)

X_train_val, X_test, y_train_val, y_test = train_test_split(X, y, test_size=0.2)
```

2. Trening sieci do regresji w schemacie walidacji krzyżowej, zapisanie najlepszych modeli na zbiorach walidacyjnych

3. Implementacja wrappera do modelu regresora pytorch

4. Ocena istotności cech modelu regresji z wykorzystaniem „permutation feature importance”

5. Wyznaczenie i wizualizacja PDP dla najistotniejszych cech:

```
from sklearn.inspection import PartialDependenceDisplay

features = [1,16,17]
PartialDependenceDisplay.from_estimator(clf, X_val, features)

features = [(1,16)]
PartialDependenceDisplay.from_estimator(clf, X_val, features)

PartialDependenceDisplay.from_estimator(clf, [1], kind='both')
```

Zadanie: Oceń rodzaj zależności zmiennej objaśnianej w zadaniu regresji od najważniejszych zmiennych objaśniających z wykorzystaniem PDP.

Zadanie 3: LIME dla „toy set”

Cel: Ocena wagi cech z wykorzystaniem metody LIME

1. Studenci implementują kod do generowania danych dla modelu wyjaśnialnego LIME, w oparciu o dane i model regresji z Zadania 2:

```
pert_factor = 0.3
num_of_perturbations = 1000
stds = X_train.std(dim=0)

X_to_explain = X_test[0]

noise = torch.randn((X_to_explain.shape[0], num_of_perturbations))*stds.unsqueeze(1)
perturbations = X_to_explain.unsqueeze(1) + noise

perturbations = perturbations.swapaxes(0,1)
outputs = model(perturbations).detach().cpu().numpy()

perturbations = perturbations.detach().cpu().numpy()
sample = X_to_explain.detach().cpu().numpy()
distances = np.exp(-np.linalg.norm(perturbations - sample,axis=1))
```

2. W oparciu o tak przygotowane dane należy wytrenować model liniowy metodą ważonej sumy kwadratów z wykorzystaniem biblioteki statsmodels

3. Ocenic jakość wytrenowanego modelu liniowego w oparciu o wartość R^2 , zwróconą w wynikach treningu

4. Ocenic wagi cech w oparciu o wartości statystyki t, zwrócone w wynikach treningu

5. Wagi cech w pkt. 4 zostały wyznaczone dla pojedynczego przypadku ze zbioru X_{test} . Powtarzając rachunki z pkt. 2 – 4 należy wyznaczyć średnie wagi cech na zbiorze testowym i porównać te wagi z wagami zwróconymi metodą permutation feature importance.

Zadanie: Wykonaj ocenę istotności cech metodą LIME w oparciu o zbiór „toy set” i porównaj tą ocenę z oceną metodą „permutation feature importance”.

Zadanie 4: Wartości Shapleya dla „toy set”

Cel: Ocena wagi cech z wykorzystaniem wartości Shapleya

1. Studenci implementują kod do generowania danych do wyznaczenia wartości Shapleya, w oparciu o dane i model regresji z Zadania 2:

```
X_to_explain = X_test[0]

means = X_train.mean(dim=0)

S = torch.arange(20) # indeksy cech dla wektora cech o długości 20
N = len(S)
```

```
K = 5 # liczba podzbiorów do wylosowania ze zbioru indeksów
```

```
m_values = torch.distributions.Binomial(N, 0.5).sample((K,)).int()
```

```
subsets = []
```

```
for m in m_values:
    idx = torch.randperm(N)[:m]
    subsets.append(S[idx].tolist())
```

```
print(subsets)
```

```
masks = torch.ones((K, N), dtype=torch.int)
```

```
for i, subset in enumerate(subsets):
    if len(subset) > 0:
        masks[i, subset] = 0
```

```
print(masks)
```

Zgodnie z metodyką oceny wagi cech z wykorzystaniem wartości Shapleya, składowe `X_to_explain` będą maskowane z wykorzystaniem masek `masks` – wartości 0 w `masks` spowodują zastąpienie odpowiednich składowych `X_to_explain` przez składowe z `means`.

2. Implementacja obliczeń wartości Shapleya dla `X_to_explain`:

```
for feature_id in range(X_to_explain.shape[0]):

    masks_with_feature = masks.clone()
    masks_with_feature[:, feature_id] = 1
    permutations = X_to_explain * masks_with_feature + means * (1 - masks_with_feature)
    outputs_with_feature = model(permutations).detach().cpu().numpy().squeeze()

    masks_without_feature = masks.clone()
    masks_without_feature[:, feature_id] = 0
    permutations = X_to_explain * masks_without_feature + means * (1 - masks_without_feature)
    outputs_without_feature = model(permutations).detach().cpu().numpy().squeeze()

    shap_value = np.mean(outputs_with_feature - outputs_without_feature)
    print(shap_value)
```

3. Ocenic wagi cech w oparciu o wartości Shapleya

4. Wagi cech w pkt. 3 zostały wyznaczone dla pojedynczego przypadku ze zbioru `X_test`. Powtarzając rachunki z pkt. 2 – 3, wybierając odpowiednio dużą wartość `K`, należy wyznaczyć średnie wartości Shapleya cech na zbiorze testowym i porównać te wagi z wagami zwróconymi metodą `permutation feature importance` i metodą `LIME`.

Zadanie: Wykonaj ocenę istotności cech metodą SHAP w oparciu o zbiór „toy set” i porównaj tą ocenę z oceną metodą „permutation feature importance” i metodą LIME.

Zadanie 5: XAI dla rzeczywistego zbioru danych

Cel: Wyjasnienia modelu zamkniętoskrzynkowego dla rzeczywistego zbioru danych

Studenci wykorzystują metody z zadań 1-4 do przygotowania wyjasnień dla modelu wytrenowanego na rzeczywistym zbiorze danych (np. pobranego z openml)

Zadanie: Wytrenuj i wyjaśnij klasyfikator/regresor dla rzeczywistego zbioru danych.